What is Bagging?

Bagging is an ensemble learning technique that improves the stability and accuracy of machine learning models by:

1. Creating many subsets of the training data (using sampling *with replacement*, called bootstrap samples).
2. Training a separate model on each subset.
3. Aggregating the results (by averaging for regression, majority voting for classification).

How Bagging Works — Step by Step

1. Bootstrap Sampling
    - Suppose you have a dataset of 1,000 rows.
    - Bagging randomly samples (with replacement) to create, say, 100 different training sets, each also of size 1,000.
    - Some rows repeat, some are left out.
2. Train Base Models
    - A separate model (e.g., decision tree) is trained on each sample.
3. Aggregate Predictions
    - For classification: Each model predicts a class → the majority class is chosen.
    - For regression: Each model outputs a number → predictions are averaged.

Example (Classification)

Dataset → Predict if a person has diabetes (Yes/No).

- Train 5 decision trees with Bagging:
    - Tree 1 → Yes
    - Tree 2 → Yes
    - Tree 3 → No
    - Tree 4 → Yes
    - Tree 5 → Yes

Final Bagging Prediction = Yes (majority vote) ✅

Why Bagging Helps

- Reduces variance → If one model overfits, others may not.
- Stabilizes predictions → Ensemble smooths out noise.
- Works especially well with unstable learners (like decision trees).

Real Example

- The most famous bagging algorithm is Random Forest (uses Bagging + feature randomness).

What is Boosting?

Boosting is an ensemble learning technique that builds models sequentially (one after another), where each new model focuses on the mistakes (errors) of the previous ones.

Instead of training models independently like Bagging, Boosting trains them in a chain, correcting errors as it goes.

How Boosting Works — Step by Step

1. Start with a weak learner
    - Usually a simple model, like a shallow decision tree (a “stump”).
    - It makes predictions but also makes errors.
2. Assign weights to data points
    - Initially, all samples have equal weight.
    - Misclassified points get higher weights (more importance) in the next round.
3. Train next model on weighted data
    - The new model tries harder to correctly classify the “hard” cases.
4. Combine models
    - Each model’s vote is weighted based on its accuracy.
    - Final prediction = weighted majority vote (classification) or weighted sum (regression).

Example (Classification)

Imagine predicting if emails are Spam (Yes/No).

- Model 1 → predicts 70% correct, but misclassifies “tricky” emails.
- Model 2 → focuses more on those tricky emails → improves.

- Model 3 → further fixes errors.
- Final prediction = weighted combination of Models 1 + 2 + 3.

Boosting “boosts” weak learners into a strong model.

Popular Boosting Algorithms

- AdaBoost (Adaptive Boosting) → adjusts weights of samples.
- Gradient Boosting → reduces errors step by step by optimizing a loss function.
- XGBoost / LightGBM / CatBoost → advanced, faster, highly popular for Kaggle competitions.

Why Boosting Helps

- Great for reducing bias (makes weak learners strong).
- Often achieves state-of-the-art accuracy.
- Works well with imbalanced datasets.

Boosting algorithms are a family of **ensemble learning methods** that combine multiple **weak learners** (usually shallow decision trees) to create a **strong predictive model**.
They work by **iteratively improving** model performance — each new model focuses more on the samples that were **misclassified** by previous ones.

Here are the **main types of boosting algorithms** you should know:

---

1. AdaBoost (Adaptive Boosting)

- **Idea:** Adjusts the weights of data points after each iteration.
  Misclassified samples get **higher weights**, so the next weak learner focuses on them.
- **Base learner:** Usually a decision stump (a tree with one split).
- **Library:** Available as AdaBoostClassifier in sklearn.
- **Pros:** Simple and interpretable.
- **Cons:** Sensitive to noisy data and outliers.

Example:

```
from sklearn.ensemble import AdaBoostClassifier

model = AdaBoostClassifier(n_estimators=100)
```

2. Gradient Boosting

- **Idea:** Instead of adjusting weights, it **fits new models to the residual errors** (gradients) of previous models.
- It's based on **gradient descent** optimization — minimizes the loss function step by step.
- **Base learner:** Decision tree.
- **Pros:** High accuracy.
- **Cons:** Slower to train compared to AdaBoost.

Example:

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)
```

3. XGBoost (Extreme Gradient Boosting)

- **Idea:** Optimized version of Gradient Boosting.
- Uses **regularization** (L1 & L2) to prevent overfitting.
- Extremely efficient and supports **parallel computation**.
- Widely used in Kaggle competitions.

Example:

```
from xgboost import XGBClassifier

model = XGBClassifier(n_estimators=200, learning_rate=0.05)
```

4. LightGBM (Light Gradient Boosting Machine)

- **Idea:** Designed for **speed and efficiency**.
- Uses a **leaf-wise growth strategy** instead of level-wise.
- Handles large datasets easily.
- Supports categorical features directly.

Example:

```
from lightgbm import LGBMClassifier

model = LGBMClassifier(n_estimators=200)
```

5. CatBoost (Categorical Boosting)

- **Idea:** Specially optimized for **categorical variables** — no need for label encoding or one-hot encoding.
- Handles overfitting well and is **easy to tune**.

- Works efficiently with smaller datasets too.

Example:

```
from catboost import CatBoostClassifier

model = CatBoostClassifier(iterations=200, learning_rate=0.1, verbose=0)
```

Summary Table

Algorithm	Key Concept	Strengths	Limitations
AdaBoost	Reweights misclassified samples	Simple, interpretable	Sensitive to noise
Gradient Boosting	Fits to residuals	High accuracy	Slow training
XGBoost	Optimized GBM + Regularization	Fast, regularized	Complex tuning
LightGBM	Leaf-wise, efficient	Handles large data	Can overfit small data
CatBoost	Native categorical support	Easy, powerful	Slightly slower

Bagging vs Boosting

Feature	Bagging	Boosting
Training	Parallel (independent models)	Sequential (models depend on each other)
Goal	Reduce variance (stability)	Reduce bias (make weak learner strong)
Example	Random Forest	AdaBoost, XGBoost

Ensemble Learning with the Pima Indians Diabetes Dataset

We'll:

1. Load the dataset
2. Split into train/test
3. Apply **single models** (Decision Tree, Logistic Regression)
4. Apply **Ensemble methods** (Bagging, Random Forest, Boosting, Stacking)
5. Compare results

What is Stacking?

Stacking is an **ensemble learning technique** where we combine predictions from **multiple different models (base learners)** using a **meta-model** (also called a blender).

Unlike Bagging/Boosting (which use many versions of the same type of model, e.g., trees), **Stacking combines different algorithms** to leverage their strengths.

How Stacking Works — Step by Step

1. **Choose base learners**
    - Example: Logistic Regression, Decision Tree, SVM, KNN.
    - Each model learns patterns from the same dataset.
2. **Train base models**
    - Each model makes predictions on the training data.
3. **Generate meta-features**
    - The predictions of base models (not raw data) become inputs for the **meta-model**.
4. **Train meta-model (blender)**
    - Example: A Logistic Regression model learns how to best combine predictions from all base learners.
5. **Final prediction**
    - For new data, base learners predict first → meta-model combines → final output.

Example

Let's say we want to predict if a patient has diabetes:

- Base learners predict:
    - Decision Tree → Yes
    - SVM → No
    - KNN → Yes
- Meta-model sees [Yes, No, Yes] → learns that "if 2 say Yes, final = Yes".
- Final prediction = **Yes** ✅

Why Stacking Helps

- Combines **diverse models** (linear + non-linear).

- Often more powerful than any single model.
- Reduces both **bias** and **variance**.

Example in Practice

- Base models: Logistic Regression, Random Forest, Gradient Boosting.
- Meta-model: Logistic Regression.
- Library: scikit-learn has StackingClassifier and StackingRegressor.

Example (Python):

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Base models
base_learners = [
    ('dt', DecisionTreeClassifier(max_depth=3)),
    ('svm', SVC(probability=True))
]

# Meta-model
meta_model = LogisticRegression()

stacking = StackingClassifier(estimators=base_learners, final_estimator=meta_model)
stacking.fit(X_train, y_train)
```

y_pred = stacking.predict(X_test)

print("Stacking Accuracy:", accuracy_score(y_test, y_pred))

Bagging vs Boosting vs Stacking

Feature	Bagging	Boosting	Stacking
Base models	Same type	Same type	Different models
Training	Parallel	Sequential	Parallel → Meta-model
Goal	Reduce variance	Reduce bias	Combine strengths
Example	Random Forest	XGBoost	StackingClassifier

Python Code for all above Ensemble Learning Algorithms:

Ensemble Learning on Pima Diabetes Dataset

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.metrics import accuracy_score

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier, StackingClassifier

📥 Load dataset (Pima Indians Diabetes dataset from UCI/Kaggle)

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

cols = ["Pregnancies","Glucose","BloodPressure","SkinThickness","Insulin","BMI","DiabetesPedigree","Age","Outcome"]

df = pd.read_csv(url, names=cols)

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#1️⃣ Base Models
dt = DecisionTreeClassifier(random_state=42)
lr = LogisticRegression(max_iter=1000)

dt.fit(X_train, y_train)
lr.fit(X_train, y_train)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))
print("Logistic Regression Accuracy:", accuracy_score(y_test, lr.predict(X_test)))

#2️⃣ Bagging
bagging = BaggingClassifier(base_estimator=DecisionTreeClassifier(), n_estimators=50,
random_state=42)
bagging.fit(X_train, y_train)
print("Bagging Accuracy:", accuracy_score(y_test, bagging.predict(X_test)))

#3️⃣ Random Forest (Bagging + Feature Randomness)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print("Random Forest Accuracy:", accuracy_score(y_test, rf.predict(X_test)))

#4️⃣ Boosting
```

```
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)

gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train, y_train)

print("AdaBoost Accuracy:", accuracy_score(y_test, ada.predict(X_test)))
print("Gradient Boosting Accuracy:", accuracy_score(y_test, gb.predict(X_test)))

#5️⃣ Stacking
estimators = [
    ('dt', DecisionTreeClassifier(random_state=42)),
    ('lr', LogisticRegression(max_iter=1000))
]
stack = StackingClassifier(estimators=estimators,
final_estimator=RandomForestClassifier(n_estimators=50, random_state=42))
stack.fit(X_train, y_train)
print("Stacking Accuracy:", accuracy_score(y_test, stack.predict(X_test)))
```

📊 **Expected Outcome (approx. values)**

- **Decision Tree** ~ 65%
- **Logistic Regression** ~ 75%
- **Bagging** ~ 72–75%
- **Random Forest** ~ 75–78%
- **AdaBoost / GradientBoosting** ~ 75–80%
- **Stacking** ~ 78–82%